

Introduction

Every developer eventually writes a loop that hits an API a hundred times and watches it crawl. The instinct is to blame the network, or blame Python, or reach for threads because "that's what concurrency means." None of those instincts are wrong exactly — they're just aimed at the wrong problem.

The real issue in most I/O-heavy code is that the CPU is doing nothing at all. It fired off a request and is now just sitting there, waiting for bytes to come back over the wire. That distinction — "slow" versus "waiting" — is one of the more useful mental models you'll pick up as a Python developer, and it's also a favorite interview topic, because it separates people who've memorized `async def` syntax from people who understand what a coroutine is actually buying them.

This lesson walks through a real scraper: sequential requests, a naive multithreaded fix, and then the `asyncio` rewrite that gets the same speedup as threading without the memory cost. Along the way we'll cover why `async` has to propagate all the way up your call stack, what `await` really does under the hood, and where `async` falls apart completely — because it's not a universal performance upgrade, it's a fix for one very specific kind of slowness.

Problem Overview

You have a list of 100 URLs. You need to fetch all of them and process the results. The naive approach is a `for` loop: request URL 1, wait for the response, request URL 2, wait for the response, and so on until all 100 are done.

Each individual request might take under a second. But because you're doing them one after another, the total time is roughly the sum of all of them — somewhere around 100 seconds for 100 requests. The catch is that during each of those waits, your CPU isn't computing anything. It's idle. You're paying wall-clock time for work the processor isn't even doing.

The task: restructure this so the waiting overlaps instead of stacking up, without exploding memory usage or introducing race conditions.

Example

Say each request takes about 1 second to get a response.

Sequential:

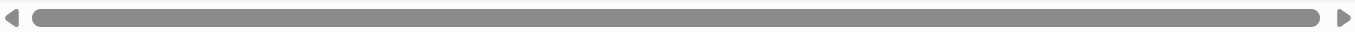
```
Request 1 → wait 1s → Request 2 → wait 1s → ... → Request 100 → wait 1s  
Total: ~100 seconds
```

Threaded (100 threads):

```
All 100 requests fired at roughly the same time, each on its own thread  
Total: ~1-2 seconds, but 100 threads' worth of memory and OS scheduling overhead
```

Async (one loop, one thread):

```
All 100 requests kicked off as tasks, the loop switches between whichever isn't currently waiting  
Total: ~1-2 seconds, no extra threads, no extra memory per request
```



The output speed of threading and async converges. The cost to get there does not.

Intuition

Here's how an engineer actually arrives at this solution rather than just memorizing it.

Start by asking: during a network request, what is the CPU doing? Nothing. It handed the request off to the OS's networking stack and now it's parked, waiting for a response to land in a buffer. That parked time is dead time — the CPU could be doing something else, but in a simple sequential script, it isn't.

The threading instinct — "give every request its own thread" — is a reasonable next step, because now the waits overlap. Thread 1 is parked waiting on request 1, thread 2 is parked waiting on request 2, and the OS switches between them. But each thread carries a real cost: a stack allocation, kernel scheduling overhead, context-switch cost. Fine for 100 requests. A disaster at 10,000.

The insight that gets you to `asyncio` is realizing you don't need 100 separate execution contexts — you need one execution context that's smart enough to notice "this task is currently waiting, let me go run a different task while I wait." That's it. That's the entire idea. One thread, one loop, and a cooperative handoff every time a task would otherwise block.

The mechanism for that handoff is `await`. It's not "pause and do nothing" — it's "I'm about to wait, so give control back to the loop and let it find something useful to do until I'm ready again."

Brute Force Solution

Idea: Fire requests one at a time, in a simple loop, waiting for each to complete before starting the next.

Algorithm: 1. For each URL in the list, call the fetch function. 2. Block until that fetch returns. 3. Store the result. 4. Move to the next URL.

Advantages: - Trivially simple to write and reason about. - No concurrency bugs possible — everything happens in a strict, predictable order.

Disadvantages: - Total time scales linearly with the number of requests, dominated entirely by network latency the CPU isn't even using. - Doesn't scale past a handful of requests before runtime becomes unacceptable.

Complexity: - Time: $O(n \times \text{latency})$ — n requests, each contributing its full round-trip time to the total. - Space: $O(1)$ beyond storing results — no extra threads or tasks in flight.

Optimal Solution

The optimal approach uses `asyncio` to run all requests on a single thread, letting the event loop switch between tasks whenever one of them is waiting on I/O.

Three pieces make this work:

- `async def` marks a pausable function.** Any function that might need to hand control back to the loop mid-execution gets defined as a coroutine with `async def`.
- `await` is the handoff point.** When you `await` something, you're telling the loop "this is where I might need to wait — feel free to run someone else until I'm ready." Because `await` can only be called inside an `async def` function, the "async-ness" of a call chain is contagious: if `fetch` is async, whatever calls `fetch` and awaits it must also be async, all the way up to `main`.
- `asyncio.gather` starts everything up front, not one at a time.** Instead of looping and waiting on each URL sequentially, you build a list of coroutine calls — none running yet — and hand the whole batch to `gather`. The loop then runs all of them concurrently, switching to whichever task isn't currently blocked on I/O.

Approach Threads used Memory overhead Time for 100 requests

Sequential	1	Minimal	~100s
Threading	100	High (per-thread stack)	Single-digit seconds
asyncio	1	Minimal	Single-digit seconds

Async gets you threading's speed without threading's memory bill, because there's only ever one thread — the loop is just very good at not sitting idle.

Python Code

```
import asyncio
import aiohttp
import time

async def fetch(session: aiohttp.ClientSession, url: str) -> str:
    """Fetch a single URL and return its response text."""
    async with session.get(url) as response:
        return await response.text()

async def fetch_all(urls: list[str]) -> list[str]:
    """Fetch every URL concurrently and return results in order."""
    async with aiohttp.ClientSession() as session:
        tasks = [fetch(session, url) for url in urls]
        return await asyncio.gather(*tasks, return_exceptions=True)

async def main() -> None:
    urls = [f"https://example.com/page/{i}" for i in range(100)]

    start = time.perf_counter()
    results = await fetch_all(urls)
    elapsed = time.perf_counter() - start

    succeeded = [r for r in results if not isinstance(r, Exception)]
    failed = [r for r in results if isinstance(r, Exception)]

    print(f"Fetched {len(succeeded)} pages in {elapsed:.2f}s")
    if failed:
        print(f"{len(failed)} requests failed: {failed[:3]}")

if __name__ == "__main__":
    asyncio.run(main())
```

Code Walkthrough

- **fetch** : Defined with `async def` because it awaits a network call. `session.get(url)` returns an async context manager; entering it and reading `.text()` are both points where the coroutine yields control back to the loop.

- **fetch_all** : Builds a list of coroutine objects — calling `fetch(session, url)` doesn't run anything yet, it just creates a coroutine. `asyncio.gather(*tasks, ...)` is what actually schedules and runs them concurrently.
- **return_exceptions=True** : Without this, the first exception raised by any task propagates immediately out of `gather`, but the other tasks keep running in the background unless explicitly cancelled. With it, exceptions are captured as objects in the results list instead, so you can inspect successes and failures side by side.
- **main** : Also `async def`, because it awaits `fetch_all`. This is the propagation rule in practice — you can't call `await` from a regular synchronous function.
- **asyncio.run(main())** : The single entry point that creates the event loop, runs `main` to completion, and tears the loop down.

Dry Run

Take three URLs instead of a hundred, each taking roughly 1 second to respond.

1. `fetch_all` builds three coroutine objects — `fetch(session, url1)`, `fetch(session, url2)`, `fetch(session, url3)` — none have run yet.
2. `asyncio.gather` schedules all three as tasks and starts running the first one.
3. Task 1 hits `await session.get(url1)`. This is a wait point, so it hands control back to the loop.
4. The loop starts task 2. Task 2 hits its own `await` and also hands control back.
5. The loop starts task 3. Same thing — it awaits and yields.
6. Now all three are "in flight" waiting on the network simultaneously. As each response arrives, the loop resumes that specific task, finishes reading its response text, and marks it done.
7. Once all three are complete, `gather` returns a list of three results, in the same order the tasks were submitted — not necessarily the order they finished in.
8. Total elapsed time: roughly 1 second, not 3, because the waits overlapped instead of stacking.

Complexity Analysis

- **Time:** $O(\max(\text{latency}) + \text{processing})$ rather than $O(n \times \text{latency})$. Since all requests are in flight concurrently, total wall-clock time is bounded by the slowest single request plus whatever CPU work is needed to process results — not the sum of every request's latency. This holds as long as the event loop and connection pool aren't themselves saturated.
- **Space:** $O(n)$ for the task list and buffered results, same as any approach that holds n items — but crucially, no per-request thread stack. A thread typically reserves megabytes of stack space; a

coroutine object is a few hundred bytes. That's the actual space win over threading, not asymptotic complexity but constant-factor memory cost.

- **Why this is correct:** the event loop guarantees that only one coroutine is *executing* Python bytecode at any instant — there's no need for locks around shared state the way there is with threads, because a context switch only happens at `await` points, which are visible in the code.

Alternative Solutions

Threading (`concurrent.futures.ThreadPoolExecutor`): Also achieves overlapping waits, and works fine for moderate request counts. Simpler to bolt onto existing synchronous code that uses `requests` instead of `aiohttp`. Loses to asyncio at scale because of per-thread memory and OS scheduling cost — 10,000 threads will bring a machine down; 10,000 coroutines won't.

Multiprocessing: Wrong tool here entirely — it solves CPU-bound problems by using multiple cores, but I/O-bound waiting doesn't need more cores, it needs better scheduling of the waiting itself.

`asyncio.as_completed` instead of `gather` : Useful when you want to process results as they arrive rather than waiting for the whole batch — same underlying mechanism, different consumption pattern.

Edge Cases

- **Empty URL list:** `gather()` with no tasks returns an empty list immediately — no special-casing needed, but worth a test so a future refactor doesn't break it silently.
- **A single request times out:** `gather` raises that exception at the point you `await` it (without `return_exceptions=True`), but does **not** cancel the other in-flight tasks — they keep running unless you cancel them yourself.
- **All requests fail:** With `return_exceptions=True`, you get a list entirely composed of exception objects — filter carefully before processing, since an unfiltered list will crash downstream code expecting strings.
- **Duplicate URLs:** Handled the same as any other URL; `asyncio` doesn't deduplicate for you, so add that logic yourself if it matters.
- **Extremely large URL list (10,000+):** Un-bounded `gather` will try to open that many connections at once; pair it with an `asyncio.Semaphore` to cap concurrency and avoid overwhelming the target server or your own connection pool.

Common Mistakes

1. **Forgetting `main` needs to be `async` too.** `await` only works inside `async def` functions — the async-ness has to run all the way up the call chain.
2. **Calling a coroutine without awaiting or scheduling it.** `fetch(session, url)` alone just creates a coroutine object; it does nothing until it's awaited or passed to `gather` / `create_task`.
3. **Looping and awaiting one at a time instead of batching with `gather`.** This defeats the entire purpose — you're back to sequential waiting, just wrapped in `async` syntax.
4. **Using `asyncio` for CPU-bound work.** A tight numeric loop has no `await` points, so it blocks the entire loop — every other task waits behind it. That's a job for threads (if I/O is also involved) or multiprocessing (if it's pure computation).
5. **Not capping concurrency for very large batches.** Firing off thousands of requests simultaneously without a semaphore can crash your own client or get you rate-limited by the server.
6. **Assuming `gather` cancels sibling tasks on the first failure.** It doesn't — the exception is raised at the await point, but the other 99 tasks keep running in the background.
7. **Mixing blocking calls (like `requests.get`) inside async code.** A synchronous blocking call inside an `async def` function still blocks the whole event loop — you need an async-native client like `aiohttp`.

Interview Questions

- What's the difference between concurrency and parallelism, and where does `asyncio` fall?
- Why doesn't `asyncio` help with CPU-bound work?
- What happens if you call an `async def` function without `await` ing it?
- How does the GIL relate to why threading doesn't help CPU-bound Python code either?
- What does `return_exceptions=True` change about `asyncio.gather`'s behavior?
- How would you limit concurrency to, say, 20 simultaneous requests out of a list of 10,000?
- What's the difference between `asyncio.gather` and `asyncio.create_task`?

Similar Problems

- **Design a rate-limited API client** — extends this pattern with a `Semaphore` to cap concurrent requests.
- **Producer-consumer with `asyncio.Queue`** — same event-loop model, different coordination pattern between tasks.
- **Implement a connection pool** — related to how `aiohttp.ClientSession` manages reusable connections under the hood.

- **Timeout handling with `asyncio.wait_for`** — a natural follow-up to the "what if one request hangs" question raised here.

Key Takeaways

- I/O-bound slowness is waiting, not computing — the CPU is idle the entire time.
- Threads fix the waiting problem too, but each one costs real memory and OS overhead.
- `asyncio` gets the same overlap as threading using a single thread, because `await` is a cooperative handoff, not a blocking wait.
- `async def` has to propagate all the way up your call stack — you can't `await` from a regular function.
- `asyncio.gather` starts a batch of tasks together rather than looping and waiting one at a time; a failure in one task doesn't cancel the rest.
- Async is not a general performance tool — it does nothing for CPU-bound work, since there's no waiting to hand off during.

Watch the Video

This lesson is easier to absorb watching the scraper actually run — sequential, threaded, and async, side by side with real timings. Watch it here: {LINK}

About the Series

This is part of **Fun with Learning Technology**, a series built around teaching real software engineering concepts the way they come up on the job and in interviews — not just syntax, but the reasoning that gets you from "why is this slow" to a solution you can defend to another engineer.

Call To Action

If this cleared up something that's been fuzzy, drop a like on the video and subscribe on YouTube — new lessons go up regularly. For deeper written breakdowns like this one, subscribe to the Substack newsletter. And if you've got a guess on what `gather` does when one task times out, or a question about where this breaks down, leave a comment — always good to see how others are reasoning through it.

Quick Review

When would you reach for asyncio over threads?

I/O-bound waiting (network/scrapper calls), not CPU-bound work — one thread runs many tasks while each awaits I/O.

What does 'await' actually do at a checkpoint?

Pauses that task, hands control back to the event loop, which runs another ready task until the awaited I/O completes.

Time cost of running 100 async I/O tasks concurrently?

Roughly the duration of the slowest single wait, not the sum — tasks overlap during idle I/O time.

Common bug: calling a blocking function inside async def

A sync call (e.g. requests.get) blocks the whole event loop, freezing every other task until it returns.

Threads vs asyncio for concurrency — key tradeoff?

Threads: OS-managed, memory-heavy, context-switch overhead. Asyncio: single thread, cooperative, needs async-aware libraries.

Interview: why doesn't async help CPU-bound code?

No waiting to overlap — the event loop can't run other tasks while one hogs the CPU; use multiprocessing instead.

Python Lesson 33: Async Programming with asyncio (async/await, event loops, tasks, and when async beats threads) — free Python cheat sheet